

Course Code: CSE 499A
Section: 15
Project Group No: 06
Group Members:
Md.Murad Ul Momin (ID: 2232360642)
Rima Saha (ID: 2132517042)
Sakia Tabassum Anonna (ID: 2221433042)
Ashraful Hossain Riyan (ID: 2232845642)

Weekly Update Report

I. INTRODUCTION

The PRIVATE SHIELD project has progressed from basic traffic collection to the preparation of a complete labelled dataset and a live IoT threat-detection pipeline. This report summarizes the integration of real and simulated traffic, automatic attack labelling, data visualization, and real-time prediction from MQTT messages.

II. SYSTEM IMPLEMENTATION

The recent implementation work established the connection between dataset preparation, attack simulation, visual validation, and continuous model-based monitoring.

A. Dataset Integration and Class Preparation

The dataset_builder.py utility was developed to combine the real N-BaIoT benchmark data with the project's simulated traffic records. Column names and feature order were aligned so both sources follow the same structure. A unified label scheme was applied: 0 for normal traffic, 1 for DoS, 2 for port scan, and 3 for Mirai-style traffic.

B. Class-Balance Verification

After merging the data sources, class counts were checked to identify imbalance among normal and attack samples. The preparation workflow supports minority-class oversampling with SMOTE when required, helping prevent the trained model from becoming biased toward the largest class.

C. Automatic Attack Labelling

The attack simulator and MQTT logger were integrated so that the active traffic mode controls the stored class label automatically. Normal messages are recorded as label 0, while DoS, port-scan, and Mirai traffic are stored as labels 1, 2, and 3. This removes the need for manual labelling and reduces the chance of inconsistent records.

D. Controlled Traffic Collection

A controlled collection sequence alternates normal and attack periods to capture realistic changes in network behaviour. The system records normal traffic, activates each attack for a fixed interval, and returns to normal operation between attacks. All observations are saved in data/labelled.csv as a single training-ready dataset.

E. Data Visualization and Validation

A visualise.py script was created to inspect the labelled dataset. It generates a histogram of packet activity by class and a scatter plot comparing packet size with flow duration. The charts make it easier to confirm that normal and malicious traffic have distinguishable distributions and to identify unusual values before model training.

F. Real-Time Detection Pipeline

The detector.py module subscribes to the devices/# MQTT topic and processes each incoming JSON message continuously. The extract_features() function converts the raw telemetry into the exact numerical feature order expected by the trained model. The Random Forest model is loaded from models/rf.pkl at startup and is used to classify every received message immediately.

G. Live Alert Generation and Testing

When the predicted class is not normal, the detector prints a real-time alert with the attack type and relevant message information. The workflow was tested by running the broker, simulated devices, attack traffic, and detector together. This confirms the complete path from MQTT data reception to feature extraction, prediction, and terminal alert generation.

FUTURE OBJECTIVES

- Measure detection, missed-attack, and false-positive rates for all attack classes.
- Store structured alert records for later evaluation and dashboard use.
- Connect the detection output to the FastAPI backend.
- Prepare the pipeline for one-command system integration.